Odds-and-Evens Game and Smart Home Alarm System

Concurrent and Distributed Programming
a.y. 2025–2026

Buda Marco Merighi Daniele Turchi Jacopo

June 13, 2026

1 Smart Home Alarm System

This exercise is about the implementation of software for a hypothetical smart home alarm control system using the actor programming model.

1.1 Identified Requirements

The central control system is placed within the context of a network of sensors. Such sensors are to be implemented as actors as well, though their main purpose is simply to propagate their activation to the control system. Indeed, sensors are always in an active state, leaving it up to the control system to determine whether to ignore or react to the firing of a given sensor, depending on its own internal state.

Similarly, a keypad should be an actor whose only purpose is relaying the intention of arming or disarming the alarm system, along with the entered PIN code.

The control system should have five states, namely `disarmed`, `exitDelay`, `armed`, `entryDelay`, and `alarm`, and should transition between them as described in the assignment, in response to incoming messages from sensors or from the keypad.

The system should allow for custom arming modes, so that even in the armed state only a subset of sensors may trigger the alarm, thus extending the solution as described in the assignment's **Optional point**. A custom arming mode can be conceptualized as a predicate on the set of sensors, identifying the active subset.

Additionally, the program should provide a way to interact with the control system or with a specific sensor from an external source that may not have a reference to the desired actor.

1.2 Design and Implementation Strategy

Apache Pekko is chosen as the framework to enable development through the actor model, along with Scala as the supporting programming language. The proposed solution

features four kinds of actors: the central `SmartHomeAlarmSystem`, a `Keypad`, a number of peripheral `SensorActor`s, and a `Home` actor which serves as a guardian for the other actor instances and allows for their interaction with the outside world. Indeed, the `Home` actor represents the root element in the actor hierarchy, as well as the entry point for the program's actor system.

All actors are implemented in the functional style (i.e. "actor as a function" philosophy), allowing for explicit management of the program's state.

Protocol The `SmartHomeAlarmSystem` actor accepts four kinds of command messages:

- `Arm`, which is handled in the `disarmed` state and contains an entered PIN code along with an optional custom arming mode.
- `Disarm`, which is handled in every state except `disarmed` and contains an entered PIN code.
- `HandleDelayEnd`, which is used internally to signal the end of a timer for exit or entry delay.
- `HandleSensorFiring`, which is handled in the `armed` state and specifies the source sensor, so that the system can check whether it should react depending on the current arming mode.

The actor specifies five `Behavior`s, implementing the five states defined in the assignment. The only difference from the description is in the ability to disarm the system while it is in the `exitDelay` state or in the `armed` state, which was not an explicit requirement.

Lastly, an exemplary scenario is added to showcase the program's functionalities. The simulation covers normal activity from a homeowner, including the arming of the system in "night mode", which responds only to a designated set of sensors, and unexpected activity from an intruder, which triggers the alarm.

2 Odds-and-Evens Game

2.1 Problem Analysis

The problem requires simulating an elimination tournament for a game of Odds-and-Evens among $N = 2^m$ players. The primary concurrent aspects are the following:

- **Concurrent Execution**: Multiple games must run simultaneously during each round. Specifically, $N/2^k$ games execute concurrently in round k.
- **Synchronization**: The tournament round structure mandates a synchronization barrier. Round $k + 1$ cannot begin until all concurrent matches in round k have concluded and the winners are identified.

- **Isolated State**: Interactions must be modeled strictly through message passing, explicitly forbidding shared memory.
- **Lifecycle Management**: Player entities must remain active across multiple rounds as long as they win, and terminate safely once eliminated.

2.2 Design and Implementation Strategy

The system is implemented in Go, leveraging goroutines for concurrent entities and channels for communication and synchronization.

System Architecture The solution consists of three primary components:

- `Organizer`: The main execution thread. He initializes 2^m players, orchestrates the tournament rounds, spawns `Referee` goroutines for each match, and aggregates the winners to form the bracket for the subsequent round.
- `Player`: Modeled as a persistent goroutine. He waits and listens on a control channel (`CommunicationChannel`) until he receives instructions and replies with game inputs on a data channel (`PlayerMoveChannel`). He maintains an active loop until a termination message is received.
- `Referee`: An ephemeral goroutine spawned by the `Organizer` to manage a single match between two players. He oversees the match protocol and reports the winner back to the `Organizer` via a dedicated result channel.

Communication Protocol Interactions are handled through typed messages (`Choice`, `Next`, `Lose`):

1. The `Referee` randomly selects one player to be the chooser and requests an Even/Odd prediction (`Choice`).
2. The `Referee` requests a number from both players (`Next`).
3. Players respond with random integers constrained by game rules.
4. The `Referee` determines the winner based on the sum of the numbers and the chooser's prediction.
5. The `Referee` sends a `Lose` message to the defeated player, causing its goroutine to exit gracefully.
6. The `Referee` writes the winning player's structure to a `result` channel.

Synchronization Barrier Synchronization between rounds is achieved implicitly through channel blocking. The `Organizer` awaits data from an array of `result` channels, one for each match in the current round. The iteration to the next round begins only when all referees have submitted their respective winners.